

Thread Test.

```
import java.util.concurrent.atomic.AtomicLong;

public class ThreadCounterLab implements Runnable {
    static AtomicLong safeCounter = new AtomicLong(0);
    static long unsafeCounter = 0;
    static boolean threadSafe;
    long instanceCounter = 0;
    long increments;

    public ThreadCounterLab(long increments) {
        this.increments = increments;
    }

    @Override
    public void run() {
        for (long i = 0; i < increments; i++) {
            if (threadSafe) {
                safeCounter.incrementAndGet();
            } else {
                unsafeCounter++;
            }
            instanceCounter++;
        }
    }

    public static void main(String[] args) throws
        InterruptedException {
```

```

if (args.length != 3) {
    System.out.println("Usage: java ThreadCounterLab +
        "<threads> <increments> <true|false>");
    return;
}
int numberOfThreads = Integer.parseInt(args[0]);
long increments = Long.parseLong(args[1]);
threadSafe = Boolean.parseBoolean(args[2]);
safeCounter.set(0);
unsafeCounter = 0;
ThreadLab[] tasks = new ThreadCounterLab(numberOfThreads);
Thread[] threads = new Thread[numberOfThreads];
long expectedCount = numberOfThreads * increments;
for (int i = 0; i < numberOfThreads; i++) {
    tasks[i] = new ThreadCounterLab(increments);
    threads[i] = new Thread(tasks[i]);
    threads[i].start();
}
for (int i = 0; i < numberOfThreads; i++) {
    threads[i].join();
}

```

```

long staticCount;
long nonStaticTotal = 0;
if (threadSafe) {
    staticCount = safeCounter.get();
} else {
    staticCount = unsafeCounter;
}
for (int i = 0; i < numberOfThreads; i++) {
    nonStaticTotal += tasks[i].instanceCounter;
}
long absoluteDifference = Math.abs(staticCount - nonStaticTotal);
double percentageDifference;
if (nonStaticTotal != 0) {
    percentageDifference = (absoluteDifference * 100.0) /
        nonStaticTotal;
} else if (staticCount == 0) {
    percentageDifference = 0.0;
} else {
    percentageDifference = Double.NaN;
}
System.out.println("Mode: " + (threadSafe ? "Thread-
    safe": "Unsynchronized"));

```

```

System.out.println("Number of threads: " + numberOfThreads);
System.out.println("Increments per thread " + increments);
System.out.println("Expected count: " + expectedCount);
System.out.println("Static count: " + staticCount);
System.out.println("Non-static total: " +
                    nonStaticTotal);
System.out.println("Absolute difference: " +
                    absoluteDifference);
if (Double.isNaN(percentageDifference)) {
    System.out.println("Percentage difference: Undefined");
} else {
    System.out.println("Percentage difference: %.4f%%",
                        percentageDifference);
}
}
}
}
}

```